

AJAX

Asynchronous JavaScript and XML



Round-Trip

En una navegación **Round-Trip** las interacciones del usuario llevan al navegador a **solicitar y recibir un documento HTML** completamente nuevo. El navegador es un motor para renderizar contenido HTML. Toda la lógica y los datos de la aplicación residen en el servidor.

Sus principales desventajas son que **hace esperar al usuario** mientras se solicita y carga el siguiente documento HTML, requiere una **gran infraestructura del lado del servidor** para procesar todas las solicitudes y administrar el estado de la aplicación, y necesita una gran cantidad de **ancho de banda** porque cada documento HTML debe ser autónomo.

Round-Trip: Objetivo de mejora

- ▶ El objetivo principal es lograr una **mayor fluidez...**
- ▶ Las preguntas serían:
 - ▶ ¿Cómo podríamos evitar recargar todo el tiempo la página?
 - ▶ ¿Cómo podríamos reducir la cantidad de información que se envía y recibe?
 - ▶ ¿Cómo podríamos independizar las peticiones HTTP?

AJAX: Asynchronous JavaScript and XML

- ▶ Es una técnica que permite procesar en **segundo plano** de manera **asíncrona** solicitudes HTTP al servidor.
- ▶ Sus inicios se remontan al año 90, pero su notoriedad y reconocimiento los logra en el año 2004 cuando Google lo aplica en aplicaciones como GMAIL y Google Maps.
- ▶ Su potencialidad se resume en **permitir peticiones HTTP al servidor sin tener que volver a cargar la página y solicitar fragmentos en lugar de documentos completos.**
- ▶ También conocido como el **modelo básico Round-Trip mejorado con JavaScript** que reducir el número de cambios completos en la página.

AJAX: Componentes claves

Mecanismo de Comunicación Cliente-Servidor

AJAX necesita un mecanismo que permita enviar solicitudes HTTP de manera asíncrona:

- ▶ **XMLHttpRequest (XHR):** Objeto tradicional, con una sintaxis más extensa para configurar la petición y más compleja para manejar el resultado.
- ▶ **Fetch API:** Alternativa actual basada en promesas. Simplifica el código permitiendo una gestión más clara y directa de las respuestas del servidor.

Ambos **permiten enviar datos al servidor y recibir respuestas** en varios formatos como JSON, HTML o texto.

AJAX: Componentes claves (Cont.)

Datos en Formato XML/JSON

- ▶ **XML:** Inicialmente, AJAX se basaba en XML para el intercambio de datos. **Proporcionaba una forma estructurada de representar datos** para ser fácilmente interpretados y manipulados.
- ▶ **JSON:** Hoy, JSON (JavaScript Object Notation) **ha reemplazado a XML** debido a su simplicidad, eficiencia y la facilidad con la que JavaScript puede parsearlo. JSON **transporta datos como pares de clave-valor**, lo que hace que el código sea más legible y mejore el rendimiento.

AJAX: Componentes claves (Cont.)

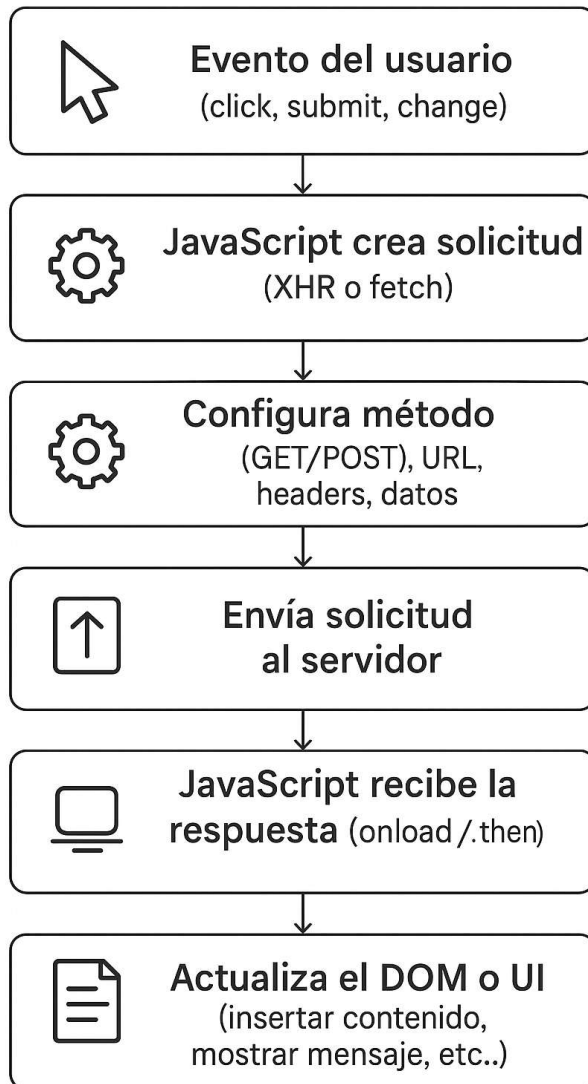
Manipulación de DOM

- ▶ Una vez recibida la respuesta del servidor, **JavaScript actualiza el contenido de la página sin necesidad de recargarla.**
- ▶ Se modifican directamente los elementos HTML afectados, manteniendo la experiencia del usuario fluida.

AJAX: Fetch API

Algunos conceptos relacionados:

- ▶ **Promesa (Promise):** Representa una **operación asincrónica que puede completarse con éxito o fallar**. Permite encadenar acciones con `.then()` y manejar errores con `.catch()`.
- ▶ **async/await:** Una sintaxis más clara para trabajar con promesas. **async convierte una función en asincrónica**, y **await pausa la ejecución hasta que la promesa se resuelve**.



AJAX: Flujo de ejecución

AJAX: Flujo de ejecución

1. **Evento del usuario:** Puede ser un clic, el envío de un formulario o un cambio en un campo. Este evento dispara la ejecución del código AJAX.
2. **JavaScript crea solicitud:** Se crea un objeto XMLHttpRequest o se utiliza fetch() para iniciar la comunicación.
3. **Configuración de la solicitud:** Se establece el método HTTP (GET o POST), la URL de destino, los encabezados y los datos necesarios.
4. **Envío de solicitud al servidor:** JavaScript realiza la petición, que se transmite vía HTTP al servidor.

AJAX: Flujo de ejecución (Cont.)

5. **El servidor procesa y responde:** El backend recibe la solicitud, ejecuta su lógica (por ejemplo, acceder a una base de datos), y devuelve una respuesta.
6. **JavaScript recibe la respuesta:** Se captura con `.then()` si se usa `fetch`, o con el evento `onreadystatechange` si se usa `XHR`.
7. **Actualización del DOM o UI:** Se modifican partes de la página según los datos recibidos, sin recargar toda la vista.

AJAX: Ventajas

- ▶ **Interactividad mejorada:** Permite crear interfaces más dinámicas y responsivas. Los usuarios reciben retroalimentación inmediata sin tener que esperar recargas completas.
- ▶ **Reducción de carga en el servidor y ancho de banda:** Solo se transmiten los datos necesarios, en lugar de recargar páginas enteras.
- ▶ **Experiencia de usuario continua:** Actualiza partes específicas de una página sin interrumpir al usuario.
- ▶ **Mayor velocidad de la aplicación:** Menos datos transmitidos y procesados generan tiempos de respuesta más rápidos.
- ▶ **Desarrollo asíncronico:** La página puede seguir funcionando mientras se procesan solicitudes, mejorando la eficiencia general.

AJAX: Desventajas

- ▶ **Problemas de SEO:** El contenido cargado dinámicamente no siempre es indexado por motores de búsqueda sin ajustes adicionales.
- ▶ **Complejidad en la depuración:** La asincronía y la interacción cliente-servidor hacen más difícil el diagnóstico de errores.
- ▶ **Gestión del historial del navegador:** Cambios dinámicos no generan nuevas URLs, dificultando el uso del botón "atrás".
- ▶ **Problemas de compatibilidad:** Algunas diferencias entre navegadores pueden afectar el comportamiento de AJAX.
- ▶ **Riesgos de seguridad:** Las solicitudes deben validarse y sanitizarse correctamente para evitar ataques como inyección o XSS.

Ejemplo del objeto XMLHttpRequest

The background of the slide features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the right side and bottom of the slide, creating a modern, dynamic aesthetic.

```
const form = document.querySelector('#miFormulario');
form.addEventListener('submit', function (e) {
    e.preventDefault();
    const datos = new FormData(form);
    const xhr = new XMLHttpRequest();
    xhr.open('POST', './procesar.jsp', true);
    xhr.onreadystatechange = function () {
        if (xhr.readyState === 4 && xhr.status === 200) {
            document.querySelector('#resultado').innerHTML = xhr.responseText;
        }
        else if (xhr.readyState === 4 && xhr.status !== 200) {
            console.error('Error en la solicitud:', xhr.statusText);
        }
    };
    xhr.send(datos);
});
```

Ejemplo de Fetch API




```
const form = document.querySelector('#miFormulario');
form.addEventListener('submit', async function (e) {
  e.preventDefault();
  const datos = new FormData(form);
  try {
    const response = await fetch('./procesar.jsp', {
      method: 'POST', body: datos
    });
    if (!response.ok) {
      const text = await response.text();
      throw new Error(text || response.statusText);
    }
    const resultado = await response.text();
    document.querySelector('#resultado').innerHTML = resultado;
  } catch (error) {
    console.error('Error en la solicitud:', error);
  }
});
```

